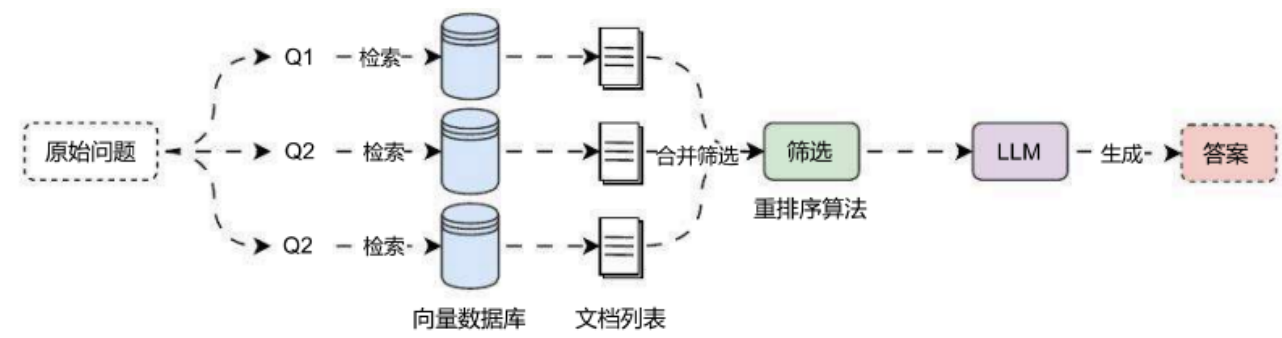


Rerank

Rerank 作为 RAG 模型中在 retrieve 和 generation 之间的一个重要环节，主要负责在大范围检索完成后对候选文档进行再精排序，从而提升最终大模型生成结果的质量和关联性。



Rerank 的作用：

- 提升检索结果相关性：
 - RAG粗排 返回的文档质量和相关性可能较差，rerank 采用更精细的语义匹配模型，过滤掉与用户问题相关性较低的文档，以及噪声和不相关的信息。
- 复杂语义理解：
 - 帮助大模型更好地理解 and 利用检索到的信息，强化相关文档的影响，从而提升生成结果的相关性和准确性。
- 降低生成模型负担：
 - RAG 检索到的文档数量较多，通过 Rerank 能较少输入文档数量，缩短上下文长度。

1. 引言

📖 相关论文：《Large Language Models for Information Retrieval: A Survey》

1.1 监督式重排（Supervised Reranking）

🍰 一句话理解：LLM 在预训练时就像个“博览群书但没当过裁判”的学者——知识渊博，却缺少 rerank 意识。监督式重排就是给它“岗前培训”：用任务相关的排序数据集（如 MS MARCO passage ranking，每条数据都标注了相关✅/不相关❌）进行微调，教会它衡量查询与文档的相关性、看懂重排任务的“考题”。

🧩 三大流派一览

按模型架构划分，监督式重排有三条技术路线，可以想象成三种不同的“评委”：

流派	形象比喻	打分方式
Encoder	🔍 阅卷老师：通读全文后直接给分	[CLS] 向量 → 线性层 → 分数
Encoder-Decoder	⚖️ 是非判官：只回答 true / false	生成标记的 logits → softmax → 分数
Decoder	💬 评论家：读完后“脱口而出”评价	解码器最后一层向量 → 分数

1.1.1 Encoder 流派 —— “阅卷式”打分

工作原理 把查询和文档**拼接成一张试卷**送入编码器：

```
[CLS] query [SEP] document [SEP]
```

模型通读后，把 [CLS] 位置的表示向量（相当于"全文中心思想"）送入**线性层**，直接输出相关性分数；再用**交叉熵损失**优化，让模型学会判断查询与文档是否"门当户对"。

✦ **优点：** 站在巨人（BERT 等预训练模型）的肩膀上，**简单微调即可上岗**，简单高效。

1.1.2 Encoder-Decoder 流派 —— "判官式"裁决

工作原理： 把重排包装成一个**生成任务**：查询 + 文档作为输入，训练模型生成一个特定标记（如 `true` / `false`）来表示相关性——就像判官只需落锤宣判"有关 / 无关"。

推理阶段则更精细：取生成标记的 **logits**，过一遍 **softmax 函数**，把"判决的底气"量化成相关性分数。

✦ **优点：** 借助生成任务的形式，能学到查询与文档之间**更复杂的语义关系**。

1.1.3 Decoder 流派 ✦ —— "评论家式"输出

工作原理： 使用 `decoder-only` 的 LLM（GPT 家族的架构）做重排，通过**解码器最后一层**的表示向量计算相关性分数；还可以请"外援"——利用现有的 **ranking 算法**（如 Cohere 等）辅助训练。

✦ **优点：** 解码器模型本就是**生成任务的王者**，能产出高质量的相关性标记。当今主流大模型几乎都是这条路线，生态红利最大。

🇨🇳 代表方法速查表（2026 视角）

流派	经典开山之作	2026 主流选手	备注
Encoder	monoBERT	BGE-Reranker-v2-m3	基于 XLM-RoBERTa，中文/多语言 RAG 场景的常青树，轻量好部署
Encoder-Decoder	monoT5 / RankT5	mxbai-rerank-v2 系列	T5 路线的延续，英文/多语言表现稳健
Decoder	RankLLaMA	Qwen3-Reranker 系列、Jina-Rerank-v3	当前性能天花板，语义理解强、支持长文本，已成 RAG 重排首选

💡 **选型口诀：** 轻量上线选 Encoder（BGE），追求精度上 Decoder（Qwen3-Reranker），硬件富裕直接 4B/8B 大杯。

🧠 记忆锚点

缺啥补啥 —— 预训练缺 `rerank` 意识 → 排序数据集微调

三种评委 —— 阅卷老师(Encoder) / 是非判官(Enc-Dec) / 评论家(Decoder)

打分位置 —— [CLS]向量 / `true-false logits` / 最后一层向量

时代趋势 —— BERT 起家 → T5 过渡 → Decoder-only 称王

1.2 无监督重排（Unsupervised Reranking）

一句话理解：大模型参数量激增后，微调成了"奢侈品"。无监督重排干脆**不动模型、只动嘴皮**——通过 **prompt 工程**直接调用 LLM 的语言天赋，不依赖标注数据，用提示词引导模型评估查询与文档的相关性、生成相关性评分。

按"出题方式"不同，可分为三种考法：

方法	一句话比喻	每次看几篇文档
Pointwise	单独面试：一次只问一篇	1 篇
Listwise	集体排队：全员站好报顺序	一批
Pairwise	二选一 PK：俩俩擂台赛	2 篇

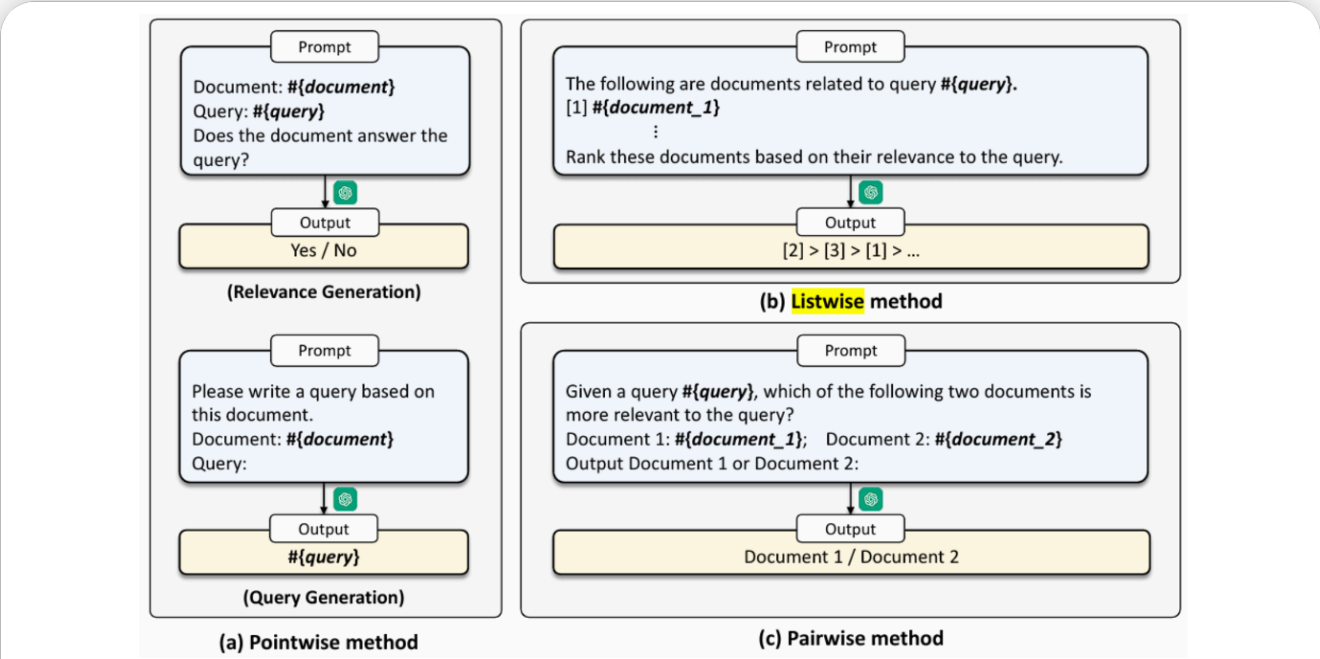


Fig. 6. Three types of prompting-based reranking methods: (a) pointwise methods that consist of relevance generation (upper) and query generation (lower), (b) listwise methods, and (c) pairwise methods.

1 Pointwise —— 单独面试

每次只评估单个文档与查询的相关性，再按分数重排。又细分两条路：

A 相关性生成（Relevance Generation）—— 直球提问

直接问 LLM：“这篇文档能回答这个查询吗？”要求输出“真”/“假”标签，再基于生成标签的 logits 计算相关性分数，通常用 softmax 算出“是”与“否”的概率：

$$f(q, d) = \frac{\exp(S_Y)}{\exp(S_Y) + \exp(S_N)} \tag{2}$$

其中 S_Y 、 S_N 分别是“真”/“假”的 log-likelihood 分数——可以理解成模型说“Yes”时的底气有多足。

B 查询生成（Query Generation）—— 反向推理

让 LLM 基于 document 反向生成一个预测 query，再看真实查询在该文档条件下的 log-likelihood 有多高——“如果这篇文档能自然地‘长出’你的问题，那它们一定很配”：

$$\text{score} = \frac{1}{|q|} \sum_i \log p(q_i \mid q_{<i}, d, \mathcal{P}) \tag{4}$$

符号速记：|q| = query 的 token 数；d = document；P = 预测 prompt。

经典案例：Co-Prompt (Discrete Prompt Optimization via Constrained Generation for Zero-shot Re-ranker)

核心问题: `prompt` 写得好不好, 直接决定重排效果。那能不能让机器自己找最优 `prompt` ?

$$\rho^* = \arg \max_{\rho} \mathbb{E}_{(d_i, q_i) \in D} [P(q_i \mid d_i, \rho)] \tag{6}$$

其中 `D` 包含所有用户 `query` 及对应检索到的相关 `document` 。

解法: 鉴别器 + 生成器的"双人舞", 基于鉴别器的条件生成方法, 遵循贝叶斯公式:

$$P(\rho_t \mid D, \rho_{1:t-1}) \propto P_{M_D}(D_s \mid \rho_{1:t}), P_{M_G}(\rho_t \mid \rho_{1:t-1}) \tag{7}$$

两位主角分工明确:

角色	身份	职责
生成器 M_G	decoder-only 大模型	像写诗一样逐 token "写" prompt
鉴别器 M_D	zero-shot 重排器	当评委, 衡量 prompt 能否指导大模型生成好的 query

鉴别器的打分标准 (query-document 对的相关性期望) :

$$P_{M_D}(D_s \mid \rho) = \mathbb{E}_{(d_i, q_i) \in D_s} [P * M_D(q_i \mid d_i, \rho)] \tag{8}$$

⚡ 效率技巧: 直接对整个词表的所有 token 算一遍太烧时间, 所以生成器只从鉴别器衡量过的 **prompt** 中采样, 并用 **beam search** 选取每一轮的 token——相当于"评委初筛 + 选手复赛", 大幅剪枝。

Algorithm 1: Co-Prompt: a beam search-based prompt generation algorithm with a discriminator and a generator. D_s : document-query pairs, B : beam width, L : maximum prompt length, N : the number of final prompts, $\mathcal{V}$: vocabulary set

```

Require:  $D_s, B, L, \mathcal{V}$ 
begin
     $P_1 \leftarrow \{\text{Start-Token}\}$ 
    for  $t = 1, \dots, L$  do
         $P_{t+1} \leftarrow \emptyset$ 
        foreach  $\rho_{1:t} \in P_t$  do
             $S_{t+1} \leftarrow \underset{K=B, \rho_{t+1} \in \mathcal{V}}{\text{top}K} P_{M_G}(\rho_{t+1} \mid \rho_{1:t})$ 
             $P_{t+1} \leftarrow P_{t+1} \cup \{\rho_{1:t+1} \mid \rho_{1:t} \oplus \rho_{t+1} \in S_{t+1}\}$ 
        end
         $P_{t+1} \leftarrow \underset{K=B, \rho_{1:t+1} \in P_{t+1}}{\text{top}K} P_{M_D}(D_s \mid \rho_{1:t+1})$ 
    end
     $P \leftarrow \cup_{t \in [1, L]} P_t$ 
     $R \leftarrow \underset{K=N, \rho \in P}{\text{top}K} P_{M_D}(D_s \mid \rho)$ 
    return  $R$ 
end
    
```

2 Listwise —— 集体排队报数

工作原理 把 `query` 和一批检索到的 `document` 一起塞进 `prompt`，直接要求 LLM 对其排序，输出形如 `[2] > [3] > [1] > ...` 的全局顺序。由于 LLM 上下文长度有限，会借用 `Long-context` 技巧（如滑动窗口、分批次排序）。

✨ **优点**：能同时考虑多个文档的相关性，生成更全局的重排结果；用 GPT-4 这类强模型效果出色。

⚠️ **三大缺点（重点记忆）**

1. 🧠 **挑模型**：只有很大的模型（如 GPT-4）才能取得良好性能——小模型排不明白；
2. 🔄 **顺序敏感**：对 `document` 在 `prompt` 中的出现顺序很敏感，文档随机摆放时，效果甚至差于 **BM25**；
3. 🚧 **难并行**：滑动窗口大小限制了单次可排序的文档数，相邻窗口间的依赖又限制了**并行推理**。

📖 经典案例：Zero-Shot Listwise Document Reranking with a Large Language Model

用如下 `prompt` 让 LLM 输出按相关性排好的 `passage id`，并用滑动窗口化解输入长度限制：

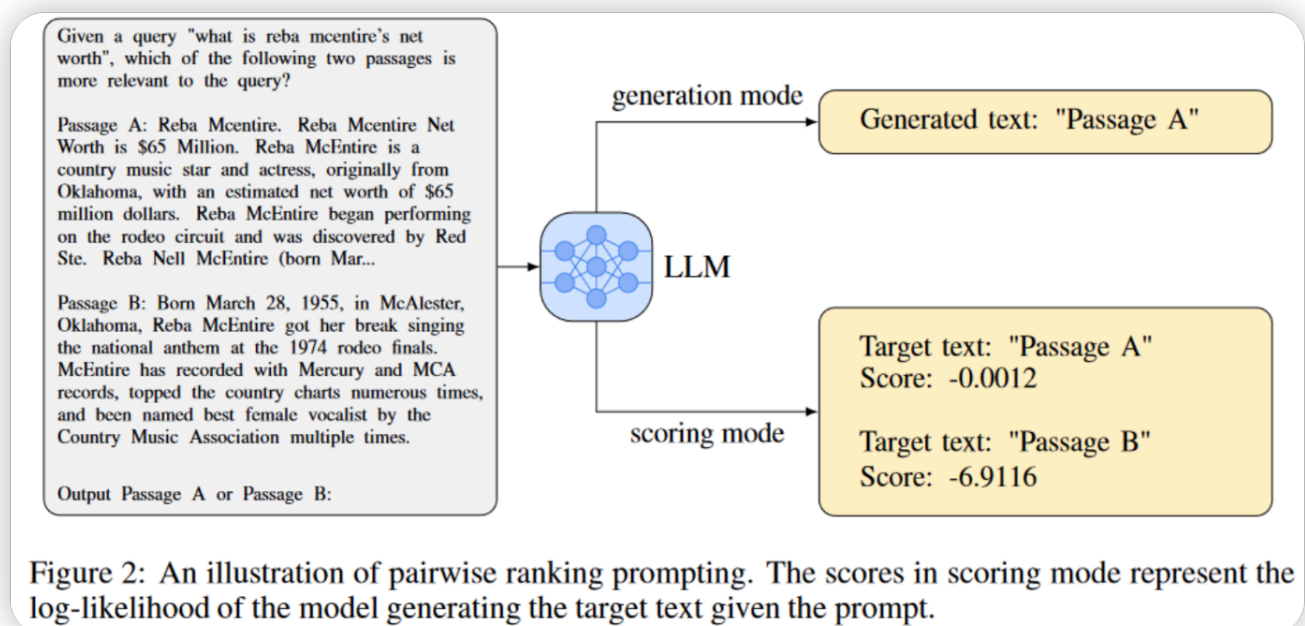
```
Passage1 = {passage_1}
...
Passage10 = {passage_10}
Query = {query}
Passages = [Passage1, ..., Passage10]
Sort the Passages by their relevance to the Query.
Sorted Passages = [
```

3 Pairwise —— 俩俩擂台赛

工作原理 利用大模型天生擅长做对比的特点：输入 = 查询 + 一对文档，让模型输出“哪篇更相关”的标记。

📖 **经典案例：PRP (Pairwise Ranking Prompting)** *Large Language Models are Effective Text Rankers with Pairwise Ranking Prompting*

- 输入是三元组 $u(q, d_1, d_2)$ ，支持生成式和打分式两种输出；生成式可能跑题，所以主要用打分式；
- 🐛 **巧用 Bug**：LLM 对输入顺序敏感本是缺陷，PRP 反手将其变成校验机制——同一三元组交换顺序输两次，若两次结果相反，则认为两个 **document** 得分一样（平局判定）。



三种变体——从“海选”到“掐尖”

变体	策略	时间复杂度	比喻
PRP-Allpair	所有 document 两两全比	$O(N^2)$	全员循环赛，公平但累
PRP-Sorting	套用快排/堆排等排序算法	$O(N \log N)$	聪明的淘汰赛
PRP-Sliding-K	类似冒泡，只冒出 top K	$O(K \log N)$	只决出前几名就收工（rerank 只关心 top K，K 较小）

🔑 番外：利用 LLM 做训练数据增强

思路转换：与其让 LLM 亲自下场重排（贵且慢），不如让它当“出题老师”——生成额外训练样本来扩充有限的标注数据集，提高小模型的泛化能力和性能。

方法	增强套路
ExaRanker	用 GPT-3.5 生成检索数据的解释，再用 query-document 对的解释训练 seq2seq 排序模型生成相关性标签
InPars-Light	prompt 大模型基于 document 生成 query（无中生有造问题）
ChatGPT-RetrievalQA	反向操作：基于用户 query 合成 document（无中生有造答案）
能力蒸馏	让 GPT 生成 ranking 列表，把大模型的排序能力迁移到小模型

🧠 记忆锚点

不微调，靠嘴皮 — prompt 工程替代标注数据
三种考法 — 单独面试(Point) / 集体报数(List) / 俩俩PK(Pair)
Pointwise 两条路 — 直球问真假 / 反向猜query
Listwise 三宗罪 — 挑模型、顺序敏感、难并行
PRP 妙招 — 换序输两次，结果打架算平局
数据增强 — LLM 不下场，改当出题老师

2. Rerank 与 Embedding

Embedding 模型的目的是将文本转化成向量表示，以便直接用于计算用户问题和文档之间的相似度。

- 💣 Embedding 模型存在的问题
 - Embedding 模型只是讲文本信息压缩为固定长度的向量，可能会导致语义信息丢失、理解多义词困难、长文本语义平均化等问题。
 - 因为在用户提出问题之前就已经为文档创建了嵌入，无法理解用户问题的上下文信息。
 - Embedding 模型计算整个查询和文档之间的相似度，难以捕捉词级、句级或精确语义关系。

Rerank 是在初步检索的基础上进一步的排序，在 用户提出查询时 才运行，可以针对具体查询分析文档的含义，而非生成一个泛化的、平均化的含义。缺点是要对用户查询和相关的多个文档一起运行 Transformer 推理，消耗更长的时间。

- 🔧 Rerank 为什么精度更高
 - 不进行预计算，而是将用户查询和一个文档一起输入到 Transformer 中，能更好的捕捉两者这件语义和上下文信息。
 - Embedding 模型将用户查询和文档压缩到低维向量中，可能丢失细粒度语义。

📌 总结:

- Rerank 模型的精度更高 -> 开销更大
|--> 一般先用 Embedding 模型初步筛选, 再 Rerank (Two-Stage Retrieval)
- Embedding 模型的精度低 -> 开销更小

3. 传统的 Rerank 模型

🔪 重排模型实例: 编码器 (BERT) vs 解码器 (GPT)

一句话理解: 两位"评分裁判"的代表作——BERT 走分类打分路线 (读完直接亮分牌), GPT 走概率推演路线 (看模型"复述"查询时有多流畅)。

🔍 编码器模型 (BERT)

📖 参考论文: 《PASSAGE RE-RANKING WITH BERT》

核心理念: 把重排当成一道"二分类题"

将 **BERT-LARGE** 当作分类模型: 取 BERT 中的 [CLS] 标记 (相当于全文的"中心思想浓缩胶囊"), 送入单层神经网络获取概率输出, 再根据这些概率对文本重排序。

这与 **Cross-Encoder** 异曲同工: 把用户查询和单个文档一起交给一个 BERT 模型, 直接计算二者的相似度——不产生 embedding, 没有中间商, 相关性判断更精准。

输入格式 (拼接成一张"试卷"):

Input: [CLS] query_token_ids [SEP] doc_token_ids [SEP]

📏 长度限制: query 最长 64 token, 整个输入最长 512 token——好比答题卡格子有限, 问题要短, 文档也得裁剪。

训练方式: 微调 + 交叉熵损失

模型输出相关性打分, 对预训练好的 BERT 进行微调, 损失函数为:

$$L = - \sum_{j \in J_{\text{pos}}} \log(s_j) - \sum_{j \in J_{\text{neg}}} \log(1 - s_j) \quad (9)$$

符号速记: J_{pos} = 相关文档的索引集合; J_{neg} = 不相关文档的索引集合, 两者都来自 BM25 检索出的 top-1000 文档。

💡 直觉理解: 相关的文档分数 s_j 越接近 1 越好 (第一项), 不相关的越接近 0 越好 (第二项) ——奖励押对, 惩罚押错。

实现代码 (数据预处理: 转 TFRecord):

```
# https://github.com/nyu-dl/dl4marco-bert/blob/master/convert_msmarco_to_tfrecord.py
def write_to_tf_record(writer, tokenizer, query, docs, labels,
                      ids_file=None, query_id=None, doc_ids=None):
    query = tokenization.convert_to_unicode(query)
    # 转化 query token id, 在前面添加了 [CLS]
    query_token_ids = tokenization.convert_to_bert_input(
        text=query, max_seq_length=FLAGS.max_query_length, tokenizer=tokenizer,
        add_cls=True)

    query_token_ids_tf = tf.train.Feature(
        int64_list=tf.train.Int64List(value=query_token_ids))

    for i, (doc_text, label) in enumerate(zip(docs, labels)):
```

```

# 转换为 doc token ids, 没有添加 [CLS]
doc_token_id = tokenization.convert_to_bert_input(
    text=tokenization.convert_to_unicode(doc_text),
    max_seq_length=FLAGS.max_seq_length - len(query_token_ids),
    tokenizer=tokenizer,
    add_cls=False)

doc_ids_tf = tf.train.Feature(
    int64_list=tf.train.Int64List(value=doc_token_id))

# 数据集标签    labels_tf = tf.train.Feature(
    int64_list=tf.train.Int64List(value=[label]))

features = tf.train.Features(feature={
    'query_ids': query_token_ids_tf,
    'doc_ids': doc_ids_tf,
    'label': labels_tf,
})

```

代码要点速记：

- query 转 token id 时前面添加 [CLS] (add_cls=True) ；
- document 转 token id 时不加 [CLS] (add_cls=False) ，且最大长度 = max_seq_length - len(query_token_ids) ——先给 query 留座，剩下的空间才归 document ；
- 最终打包成 query_ids / doc_ids / label 三个特征字段。

🧠 解码器模型 (GPT)

📖 参考论文：《SGPT: GPT Sentence Embeddings for Semantic Search》

SGPT Cross-Encoder：用"语言模型的直觉"打分

通过 GPT 实现 Cross-Encoder 交叉编码器：将 query 和 document 拼接起来一起编码，然后基于获取的对数概率 (log probabilities) 计算分数——不训练分类头，直接借用 GPT 与生俱来的"语感"。

```

## https://github.com/Muennighoff/sgpt/blob/main/README.md#asymmetric-semantic-search-ce
prompt = 'Documents are searched to find matches with the same content.\nThe document "{}" is a good search result for "'
for query in queries:
    print(f"Query: {query}")
    for doc in docs:
        context = prompt.format(doc)

        context_enc = tokenizer.encode(context, add_special_tokens=False)
        continuation_enc = tokenizer.encode(query, add_special_tokens=False)
        ## 拼接 query 和 document
        model_input = torch.tensor(context_enc+continuation_enc[:-1])
        continuation_len = len(continuation_enc)
        input_len, = model_input.shape

        ## 获取对数概率
        logprobs = torch.nn.functional.log_softmax(model(model_input)[0], dim=-1).cpu()
        logprobs = logprobs[input_len-continuation_len:]
        ## Gather the log probabilities of the continuation tokens -> [continuation_len]
        logprobs = torch.gather(logprobs, 1, torch.tensor(continuation_enc).unsqueeze(-1)).squeeze(-1)
        score = torch.sum(logprobs)

```



```
## The higher (closer to 0), the more similar
print(f"Document: {doc[:20]} + '...' Score: {score}")
```

小抄：

- (a) Cross-Encoder 把 query + documents 拼一起编码，分数 = query token 的 log 概率之和；
- (b) Bi-Encoder 分开编码、可缓存文档向量，分数 = 余弦相似度。

数学推导：贝叶斯公式撑腰

给定用户查询 q 、文档集合 D ，目标是找最相关的文档 d^* ：

$$d^* = \arg \max_{d \in D} P(d \mid q) = \arg \max_{d \in D} \frac{P(q \mid d), P(d)}{P(q)} = \arg \max_{d \in D} P(q \mid d), P(d) \tag{10}$$

为什么能化简？

- $P(q)$ 对所有文档都是同一个**固定值**——分母相同，比大小时直接扔掉；
- $P(d)$ 的变化量比较小——近似当作背景板。

于是问题变成：哪篇文档最能“自然地生成”用户的查询，哪篇就最相关（即比较 $P(q \mid d)$ ）。

⚙️ **工程细节：**限制模型输出的 query 长度与原始用户 query 长度一致，并对过长的 document 进行裁剪。由于用户 query 长度固定，比较 $P(q \mid d)$ 也就更容易——**统一赛道长度，成绩才有可比性**。

🇮🇹 两种路线对比速查

维度	BERT（编码器）	SGPT（解码器）
打分依据	[CLS] 向量 → 分类头输出概率	query token 的 log 概率
理论支撑	二分类 + 交叉熵损失	贝叶斯公式： $\arg \max P(q \mid d)$
是否需训练	需微调（监督式）	可零样本（借 LM 天赋）
形象比喻	阅卷老师：读完直接给分	测谎仪：看“复述问题”顺不顺口

🧠 记忆锚点

BERT 三件套 — [CLS] 拼接 / 单层分类头 / 交叉熵微调
长度规矩 — query ≤ 64，总长 ≤ 512，先给 query 留座
SGPT 一句话 — 文档若相关，“长出”查询不费劲（比 $P(q \mid d)$ ）
贝叶斯化简 — $P(q)$ 固定扔掉， $P(d)$ 变化小忽略